

Technical Design Document (TDD)

F1tenth PoliMI

The simulator: gym, bridge and car models

Document type:	Technical Design Document (TDD)
Version:	0.01
Date:	November 8, 2026
Author:	F1tenth PoliMI team
Classification:	Public

Document History

Version	Date	Author	Changes
0.01	2026-10-25	F1tenth PoliMI team	Initial draft from `simulator.md` and `polimi_sim`

Document Approvals

Role	Name	Date	Signature
Technical Lead			
Project Manager			
Management			

Contents

1 Introduction . . . 3
- 1.1 Purpose . . . 3
- 1.2 Scope . . . 3
- 1.3 References . . . 3
- 1.4 Status Conventions . . . 3

2 Architecture . . . 3
- 2.1 Component View . . . 3
- 2.2 Components . . . 4
- 2.3 Deployment View . . . 4
- 2.4 Design Decisions . . . 5

3 Data . . . 5
- 3.1 Configuration files . . . 5
- 3.2 Model parameters . . . 6
  - 3.2.1 `lidar_model` (`config/lidar/sl450.yaml`) . . . 6
  - 3.2.2 `actuation_model` (`actuation_model.yaml`) . . . 6
  - 3.2.3 `odom_model` (`odom_model.yaml`) . . . 7
  - 3.2.4 `teleop_bridge` (`teleop_bridge.yaml`) . . . 7
- 3.3 Ideal mode . . . 7

4 Interfaces . . . 8
- 4.1 Simulator versus car . . . 8
- 4.2 Topics . . . 8

5 Requirements Traceability . . . 9

6 Testing . . . 9

7 Known Limits . . . 9

8 Open Items . . . 10

1 Introduction

1.1 Purpose

This document defines how the simulator stands in for the car: what the gym computes, which model nodes sit between the gym and the stack, how they are configured, and where the simulator still differs from the car.

1.2 Scope

- **In scope:** f1tenth_gym and f1tenth_gym_ros as used here, our `polimi_sim` package (model nodes, launch, configs), the simulator side of the car interface (`hw:=sim`).
- **Out of scope:** the stack and the car, in `TDD_F1tenth`. How to run the simulator day to day is the runbook `docs/software/simulator.md`.

1.3 References

- `RASD_F1tenth`: FR-SIM-001 to 003, NFR-POR-001.
- `TDD_F1tenth`: the car interface (its Architecture and Interfaces sections), DEC-03 to DEC-10.
- Simulation plan 01.4 (Jazzy, the JAX gym, `polimi_sim` in our fork) and plan 01.5 (camera and IMU models).
- Runbook `docs/software/simulator.md`; host shortcuts in `scripts/sim/`.

1.4 Status Conventions

[Decided] agreed **[Proposed]** proposed, waiting for the team's confirmation **[01.5]** later plan **[TBD]** open [MEASURE] replace with a measurement on the car [VERIFY] check against the real driver or firmware.

> **Today versus target.** This document describes the simulator as built today: ROS 2 Humble, f1tenth_gym and f1tenth_gym_ros `dev-humble` pinned by `GYM_REF` and `GYM_ROS_REF` in `docker/ros/Dockerfile`, `polimi_sim` in this repo, and `ackermann_mux` inside `sim_car.launch.py`. Plan 01.4 moves it to Jazzy, the JAX gym (`dev-jax`) and the bridge `dev-jazzy`, with `polimi_sim` in our fork `f1tenth_gym_polimi`; `TDD_F1tenth` DEC-08 **[Proposed]** then takes the mux out of the simulator.

2 Architecture

> **Key point.** The gym alone is too kind: a perfect scan 250 times a second, commands that act at once, a pose that is always exact. Code tuned on that breaks on the car. `polimi_sim` puts model nodes between the gym and the stack, so the stack runs against the car's topics, frames, rates and imperfections. The gym stays the source of physics and ground truth.

2.1 Component View

Read bottom-up: the stack publishes `/drive`, the mux lets teleop override it, `actuation_model` delays and limits the command as the VESC and servo would, and the gym moves the car. From the new true pose, `lidar_model` produces `/scan` and `odom_model` produces `/odom`. A static TF `base_link→laser` comes from `config/car/laser_mount.yaml`. The truth, `map→ego_racecar/base_link` and `/ego_racecar/odom`, is for evaluation only.

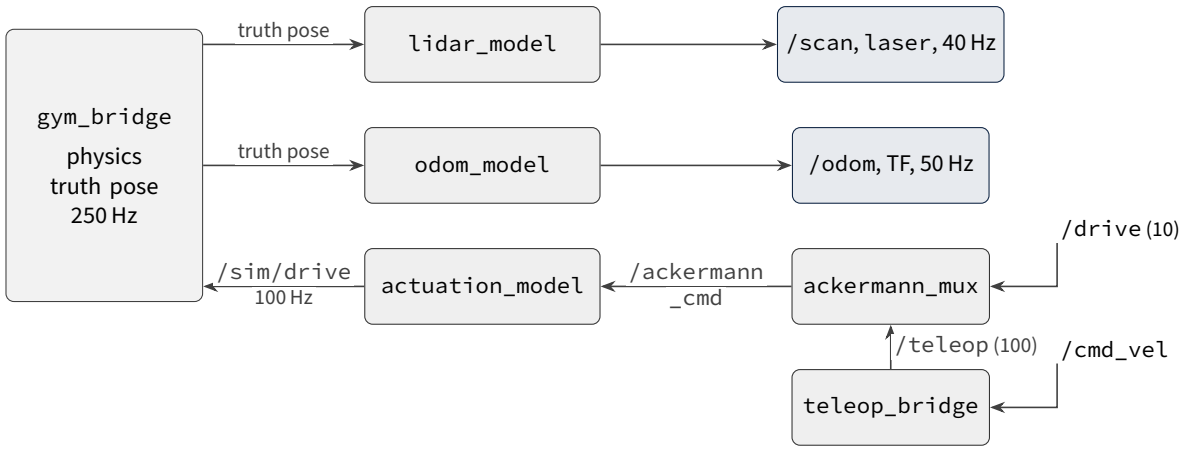


Figure 1: The simulator graph today. The gym computes where the car really is; three model nodes turn that into what the car's sensors would report and what its actuators would do. The mux is the car's own; with DEC-08 of `TDD_F1tenth` it moves to the stack's common layer.

2.2 Components

Component	Responsibility	Interfaces
f1tenth_gym	Physics: single-track model, collisions, ray caster	Python API
`gym_bridge` (f1tenth_gym_ros)	Runs the gym in ROS; truth pose and map	`/sim/drive` → truth TF, `/ego_racecar/odom`, `/map`, `/sim/scan_unused`
`lidar_model`	The SL450 as a rotating sensor with its errors	truth pose → `/scan`
`odom_model`	The car's `vesc_to_odom`, drift included	truth pose, `/sim/steering_command` → `/odom`, TF `odom`→`base_link`
`actuation_model`	The VESC and servo: delays, rates, limits, timeout	`/ackermann_cmd` → `/sim/drive`, `/sim/steering_command`
`teleop_bridge`	Keyboard and Foxglove teleop into the mux	`/cmd_vel` → `/teleop`
`ackermann_mux`	The car's mux and `mux.yaml` (today)	`/teleop`, `/drive` → `/ackermann_cmd`
`sim_car.launch.py`	Merges the configs, starts all of the above; `ideal:=true`	launch arguments

2.3 Deployment View

The simulator runs in the `arm` (Apple Silicon) and `x86` images only, never on the car (`jet`). The gym and the bridge are cloned at fixed commits during the image build; `polimi_sim` and the configs are bind-mounted like the rest of the workspace. Foxglove connects on `ws://localhost:8765`; RViz through the optional noVNC container.

2.4 Design Decisions

IDs here are local to this document; `TDD_F1tenth`'s are written with its name.

ID	Decision	Rationale / alternatives
DEC-01	Model nodes between gym and stack; the gym stays physics and truth **[Decided]**	The stack meets the car's rates, frames and errors without changing the gym. Each model can be switched off (ideal mode) to see what it changes.
DEC-02	Each thing is defined once, in its own file under `ros2_ws/config/` **[Decided]**	Map, laser, mount and gym settings are chosen independently and merged at launch. The gym's own LiDAR takes its beams, angles, ranges and mount from the same laser and mount files, so the laser is never described twice.
DEC-03	The gym's LiDAR stays on, noise-free, on `/sim/scan_unused` **[Decided]**	Its wall collision check runs on that scan. The stack's `/scan` comes only from `lidar_model`.
DEC-04	Two frame trees: truth apart from the estimate **[Decided]**	`map`→`ego_racecar/base_link` is where the car is; `odom`→`base_link`→`laser` is where it thinks it is. Nothing joins them except SLAM, localization or the debug option, so the stack cannot read the truth by accident (FR-SIM-003).
DEC-05	`lidar_model` ray-casts with the gym's own caster **[Decided]**	From the truth pose composed with the mount, in blocks along the sweep, so motion distortion comes from real poses; the map matches the gym's.
DEC-06	Ideal mode is the same graph with overrides **[Decided]**	Same nodes, topics and frames; only the imperfections go. The overrides live in `sim_car.launch.py` (`IDEAL_*`) and win over the parameter files.
DEC-07	Controllers read beam angles from each `LaserScan` **[Decided]**	`angle_min + i * angle_increment`, never hard-coded indices, so the same code runs with any laser and on the car.
DEC-08	`sim/gym.yaml` is the bridge's `config/sim.yaml` at the pinned commit, minus our keys **[Decided]**	Re-copied when `GYM_ROS_REF` is bumped; its header lists what differs.
DEC-09	The mux leaves the simulator **[Proposed]**	Follows `TDD_F1tenth` DEC-08: the fork runs from a bare clone, the stack's common layer owns the mux, and `teleop_bridge` publishes on `teleop_topic`.

3 Data

3.1 Configuration files

All under `ros2_ws/config/`, bind-mounted: no rebuild. At launch the four are merged into the config the gym gets, written to `/tmp/polimi_sim/gym_<map>_<lidar>.yaml`.

File	Holds	Chosen with
`maps/<map>.yaml`	`map_path`, start pose `sx`, `sy`, `stheta`; any other `gym_bridge` parameter overrides `sim/gym.yaml`	`simcar <map>`
`lidar/<lidar>.yaml`	The laser: rate, FOV, resolution, ranges, effects	`lidar:=<lidar>` (default `sl450`)
`car/laser_mount.yaml`	`base_link`→`laser`: x 0.27, y 0, z 0.11 m, yaw 0, roll 0 [MEASURE]; read by the car too (`TDD_F1tenth` DEC-09)	–
`sim/gym.yaml`	Everything else: topics, vehicle preset, opponents	–

Maps today: `spielberg` (default; Red Bull Ring, a closed track about 2.2 m wide) and `levine` (a corridor loop); the bridge also bundles `levine_obs`, `levine_blocked` and `blank`.

3.2 Model parameters

First guesses from datasheets and upstream defaults. `actuation_model`, `odom_model` and `teleop_bridge` defaults are in `polimi_sim/config/`; the laser's in `config/lidar/`. Live: all `lidar_model` effects, all `actuation_model` and `teleop_bridge` values, the `odom_model` error values. Fixed at start: rates, scan geometry, `range_max`, topics, frames.

3.2.1 `lidar_model` (`config/lidar/sl450.yaml`)

Parameter	Default	Meaning
`rate_hz`	40	SL450 rates: 15, 20, 25, 30, 40 Hz
`fov_deg`, `resolution_deg`	270, 0.2	1351 beams; `time_increment` $= 1/(\text{rate} \cdot 1800)$, `scan_time` $= 1/\text{rate}$: the sweep lasts 18.75 ms at 40 Hz
`blocks`	8	Groups cast from the truth pose at their own time; 1 = no motion distortion
`stamp_at`, `publish_delay`	`start`, 0.002 s	[VERIFY] against the SL450 driver
`no_return_value`	`'inf'`	REP 117 [VERIFY]
`noise_std`	0.01 m	Range noise
`dropout_prob`	0.002	Random missing beams
`grazing_dropout_prob`, `grazing_max_deg`	0.5, 80°	Loss at grazing incidence
`reflectivity_range_max`	15 m	Low-reflectivity limit
`mixed_pixel_prob`, `mixed_pixel_jump`	0.3, 0.3 m	Mixed pixels at range jumps
`seed`	0	0 = different every run

Each effect is off at 0.0. Cost: 0.1 ms per scan on the Mac, against a 5 ms budget.

3.2.2 `actuation_model` (`actuation_model.yaml`)

A command does not act at once: it travels to the VESC, the servo needs time to turn, the motor cannot change speed instantly, and the VESC stops the motor when commands stop. The node

sits where the VESC would, on a 100 Hz timer, and publishes the steering as sent to the VESC on `/sim/steering_command` (`Float64`, rad).

Parameter	Default	Meaning
`command_delay`	0.02 s	Command transport
`vesc_timeout`	0.5 s	No command for longer: speed 0, steering held [VERIFY] in the VESC app config
`steer_offset`	0.0 rad	Servo trim
`servo_delay`, `servo_rate`	0.03 s, 3.2 rad/s	Servo response
`max_steer`	0.4189 rad	Steering clamp
`accel_max`, `brake_max`	4.0, 6.0 m/s^2	Motor limits

All [MEASURE] on the car. The gym still applies its own limits afterwards (steering 3.2 rad/s, acceleration 9.51 m/s^2).

3.2.3 `odom_model` (`odom_model.yaml`)

Mirrors the car's `vesc_to_odom`: `/odom` (frame `odom`, child `base_link`) and TF `odom`→`base_link` at 50 Hz, starting from 0.

- Speed = true longitudinal speed × (1 + `speed_scale_error` 0.02) + noise `speed_noise_std` 0.02 m/s, zeroed below `speed_deadband` 0.05 m/s.
- Yaw rate = $v \tan\delta / L$, with δ the *commanded* steering and `wheelbase` L 0.3302 m (the gym's) [MEASURE]: the car's `vesc.yaml` has 0.25 (`TDD_F1tenth` T-05).
- The yaw rate reads high in corners (9% at 1.5 m/s, steering 0.25 rad): the gym's tyres slip and the formula assumes they do not. `vesc_to_odom` on the car has the same error.
- `debug_map_to_odom`: `off` (default) whenever SLAM or localization runs; `static` fixes `map`→`odom` at the true start pose, so the drift shows (about 8° per 90° corner); `truth` follows the truth like a perfect localizer, so `/scan` stays on the map while `/odom` still drifts.
- `/odom_model/reset` (`std_srvs/Empty`) zeroes `/odom` and re-pins the `static` overlay.

3.2.4 `teleop_bridge` (`teleop_bridge.yaml`)

Keyboard and Foxglove teleop speak `Twist`, the mux speaks Ackermann. `speed = linear.x`; the steering from `angular.z`, as `angular_mode: steering_angle` (rad, default) or `yaw_rate` ($\arctan(L\omega/v)$, $|v|$ never below `min_speed`); clamped to `max_steer`.

3.3 Ideal mode

`ideal:=true` keeps the nodes, topics and frames and switches the imperfections off (DEC-06).

Node	Overrides
`lidar_model`	`effects_enabled: false`, `blocks: 1`, `publish_delay: 0.0`
`actuation_model`	`command_delay`, `servo_delay`, `steer_offset`: 0.0; `servo_rate`, `accel_max`, `brake_max`: 1000.0 (the gym's own limits remain)
`odom_model`	`speed_scale_error`, `speed_noise_std`, `speed_deadband`: 0.0
`debug_map_to_odom`	`truth`, unless set on the command line

Still there in ideal mode: the 40 Hz scan with rotating-sensor time fields and `inf` beyond 25 m; the mux timeout (0.2 s), the VESC timeout (0.5 s) and the steering clamp; the `/odom` yaw drift from tyre slip.

4 Interfaces

4.1 Simulator versus car

	Gym alone	**`simcar`**	**Car**
`/scan`	250 Hz, all beams at one instant, frame `ego_racecar/laser`	40 Hz, rotating-sensor timing and motion distortion, frame `laser`	SL450 driver
Scan errors	Gaussian noise	noise, dropouts, grazing loss, 15 m low-reflectivity limit, mixed pixels	real
Command path	`/drive` straight into physics, held forever	`/drive` → `ackermann_mux` → `actuation_model`	mux → VESC
Actuation	ideal	command delay, VESC timeout, servo delay and rate, accel and brake limits	VESC, servo
Odometry	truth in `map`	speed scale error and noise, yaw rate from the commanded steering; drifts	`vesc_to_odom`
Frames	`map`→`ego_racecar/base_link`→`ego_racecar/laser`	`odom`→`base_link`→`laser`, truth tree apart	`odom`→`base_link`→`laser`
Teleop	`/cmd_vel` into physics	`/cmd_vel` → `teleop_bridge` → `/teleop` → mux	joystick → `/teleop` → mux

4.2 Topics

The stack may use only `/scan`, `/odom` and TF `odom`→`base_link`→`laser`, and publish `/drive`. It never reads `ego_racecar/*` or `/ego_racecar/odom`: that is the ground truth, which the car does not have (FR-SIM-003).

Topic	**Type**	**Direction**
`/scan`	`LaserScan`, frame `laser`, 40 Hz	out, `lidar_model`
`/odom`	`Odometry`, frame `odom`, 50 Hz	out, `odom_model`
`/drive`	`AckermannDriveStamped`	in, autonomy, mux priority 10
`/teleop`	`AckermannDriveStamped`	in, mux priority 100, from `teleop_bridge`
`/cmd_vel`	`Twist`	in, teleop
`/initialpose`	`PoseWithCovarianceStamped`	in, reset of the true pose
`/map`	`OccupancyGrid`	out
`/ego_racecar/odom`	`Odometry`, truth in `map`	out, evaluation only
`/ego_racecar/collision`	`Bool`	out, evaluation only

Topic	Type	Direction
`/ackermann_cmd`, `/sim/drive`, `/sim/steering_command`, `/sim/scan_unused`	internal	–

5 Requirements Traceability

Requirement	Component(s)	Test(s)
`FR-SIM-001`	model nodes, configs (DEC-01, DEC-02)	`ros2 topic info` and `tf2_echo` against the car interface
`FR-SIM-002`	mux, `actuation_model` timeouts; locks and AEB with `TDD_F1tenth`	AC-V1-04 in the simulator
`FR-SIM-003`	two frame trees (DEC-04)	no stack node subscribes to `ego_racecar/*`
`NFR-POR-001`	`arm` and `x86` images	the same `simcar` on both

6 Testing

Level	Scope	Tooling / gate
Unit	Model logic, plain Python without ROS, on synthetic data	pytest in `polimi_sim/test`
Comparison	Each model against the gym alone	ideal mode against the normal run; `effects_enabled false` against `/sim/scan_unused` (they match standing still except where the gym rounds beam angles to a table)
System	The stack on the simulator	AC-V1-02 and 04 in the simulator first (`RASD_F1tenth`)

7 Known Limits

Where the simulator still differs from the car, or simplifies:

- No camera, and no room with a taped track: the maps are tracks bounded by walls, which the project's environment no longer has (`RASD_F1tenth` C-001 to C-005). The LiDAR sees those walls; in the room it will see only the room. Camera and IMU models: **[01.5]**.
- Truth pose: the gym's single-track state is at the centre of gravity, published as `ego_racecar/base_link`. `polimi_sim` treats it as `base_link` (the rear axle on the car): a 0.17 m offset.
- Vehicle body parameters are the gym's `f1tenth` preset.
- The truth pose changes at 100 Hz (the physics step). `lidar_model` interpolates between steps and extrapolates up to 30 ms past the newest.
- Grazing loss and mixed pixels come from neighbouring beams, not from the map geometry.
- `lidar_model` needs the bridge on wall-clock time: `use_sim_time: False` in `sim/gym.yaml`.
- The gym's 3D car model and wheels stay on the `ego_racecar/*` frames.

- A wall hit makes the truth jump: the gym zeroes the car's heading on collision; `/odom` does not follow.
- `/initialpose` alone moves the truth, not `/odom`: wheel odometry keeps integrating, as on the car.

8 Open Items

ID	Topic	Notes
S-01	Deadman and e-stop stand-in	No USB passthrough on Docker Desktop for macOS, so no joystick in the container (`TDD_F1tenth` T-01).
S-02	Measured values [MEASURE]	Laser mount, actuation parameters and wheelbase from the car, once built.
S-03	SL450 driver behaviour [VERIFY]	`stamp_at`, `publish_delay`, `no_return_value`, and the VESC timeout.
S-04	Room maps	A room with a taped track in place of the walled tracks, with the camera model (plan 01.5).
S-05	Migration	Plan 01.4: Jazzy, the JAX gym, `polimi_sim` in `f1tenth_gym_polimi`; the mux leaves the simulator (DEC-09).